🔍 Supplier Investigation Filter Documentation

⚠️ Current Problem Analysis

The existing supplier investigation system suffers from a monolithic validation approach that makes debugging and maintenance extremely challenging.

Big Combined Filter Issue: All validation rules are bundled into a single, complex filter making it impossible to identify which specific condition failed for each supplier.

Poor Error Reporting: When validation fails, the system cannot pinpoint the exact reason, leading to frustrating debugging sessions.

✅ Two Powerful Solutions

We present two elegant approaches to solve the supplier validation problem. Both eliminate the "big combined filter" issue but use different architectural patterns.

Approach 1 - Chain of Responsibility with Functional Programming: Uses lambda expressions and Optional to create a flexible rule chain that stops at first failure.

Approach 2 - Sequential Validation with DTO: Uses traditional if-else validation with early return and structured result objects for clear success/failure reporting.

Approach 3 - Static error variable: Add a static variable in SearchFiltarationRefactor class , then at each separate filter this error message will be assigned. Why static?! so I can easily call it outside the class

All Approaches: Provide specific error messages, enable easy rule addition, and support comprehensive testing of individual validation rules.

🔗 Chain of Responsibility Pattern Overview

🎬 Real-World Analogy: Netflix Credit Card Issue

Imagine you receive a message that your Netflix subscription failed due to a credit card issue. You call your bank to resolve this problem.

You Call Bank → Robotic Voice → Operator 1 → Operator 2 → Problem Solved!

Why this works: Each operator (handler) in the chain has a specific responsibility. If they can't handle your request, they pass it to the next handler who is better equipped to solve your specific issue.

📋 How Chain of Responsibility Works

1. **Request Initiated:** A supplier validation request enters the chain
2. **Handler Processing:** Current handler examines if it can process the request
3. **Decision Point:** Handler either processes the request or passes it to the next handler
4. **Chain Continues:** Process repeats until a handler can fully process the request or chain ends

🎯 Key Benefits

🚀 Functional Programming

Uses Java 8+ lambda expressions and Optional class for clean, expressive code that's easy to read and maintain.

⚡ Performance Optimized

Chain stops at first rule failure, avoiding unnecessary validation processing for already-failed suppliers.

🔧 Easy Rule Management

Add new rules by simply adding lambda expressions to the RULES list - no need to create new classes or modify existing code.

📊 Detailed Error Reporting

Each rule provides specific, actionable error messages that pinpoint exactly why a supplier was rejected.

🧪 Individual Rule Testing

Each rule can be tested independently by accessing it from the RULES list, enabling comprehensive unit testing.

🔍 Flight-Specific Validation

Rules are tailored for flight booking scenarios: PCC validation, currency support, trip types, cabin classes, and airline preferences.

📈 Implementation Roadmap

🗓️ Phase-by-Phase Implementation

1. **Phase 1 - Interface Design:** Create the SupplierRule functional interface and define the main validation service structure.
2. **Phase 2 - Rule Migration:** Convert existing validation logic into lambda expressions in the RULES list, starting with the most critical validations.